System Verification and Validation Plan for Software Engineering

Team 20, Aeolus Software
Andy Huynh
Andreas Sideris
Humna Saeed
Christopher Ludwig
Owen Johnson

November 3, 2026

Revision History

Date	Version	Notes
Date 1	1.0	Notes
Date 2	1.1	Notes

[The intention of the VnV plan is to increase confidence in the software. However, this does not mean listing every verification and validation technique that has ever been devised. The VnV plan should also be a **feasible** plan. Execution of the plan should be possible with the time and team available. If the full plan cannot be completed during the time available, it can either be modified to "fake it", or a better solution is to add a section describing what work has been completed and what work is still planned for the future. —SS]

[The VnV plan is typically started after the requirements stage, but before the design stage. This means that the sections related to unit testing cannot initially be completed. The sections will be filled in after the design stage is complete. the final version of the VnV plan should have all sections filled in. —SS]

Contents

1 Symbols, Abbreviations, and Acronyms **iv**

2 General Information **1**
- 2.1 Summary . . . 1
- 2.2 Objectives . . . 1
- 2.3 Custom Documents . . . 2
- 2.4 Relevant Documentation . . . 3

3 Plan **4**
- 3.1 Verification and Validation Team . . . 4
- 3.2 SRS Verification . . . 5
- 3.3 Design Verification . . . 6
- 3.4 Verification and Validation Plan Verification . . . 7
- 3.5 Implementation Verification . . . 9
- 3.6 Automated Testing and Verification Tools . . . 9
- 3.7 Software Validation . . . 10

4 System Tests **11**
- 4.1 Tests for Functional Requirements . . . 11
  - 4.1.1 Configuration Tests for Functional Requirements . . . 11
  - 4.1.2 Central Hub Tests for Functional Requirements . . . 15
  - 4.1.3 Area of Combat Tests for Functional Requirements . . . 19
  - 4.1.4 User Interface Tests for Functional Requirements . . . 22
- 4.2 Tests for Nonfunctional Requirements . . . 25
  - 4.2.1 Configuration Tests for Nonfunctional Requirements . . . 25
  - 4.2.2 Area of Testing1 . . . 25
  - 4.2.3 Area of Combat Tests for Non-Functional Requirements 25
  - 4.2.4 Central Hub Tests for Nonfunctional Requirements . . . 28
  - 4.2.5 User Interface Tests for Non-Functional Requirements . . . 29
  - 4.2.6 Software Usability Tests . . . 32
- 4.3 Traceability Between Test Cases and Requirements . . . 33

5 Unit Test Description **34**

6 Appendix **36**
- 6.1 Symbolic Parameters . . . 36
- 6.2 Usability Survey Questions . . . 37

7 Appendix — Generative AI Use Disclosure **39**
7.1 AI Tools Used 39
7.2 How and Where Used 39
7.3 Verification of AI-Generated Content 39

8 Appendix — Reflection **40**

List of Tables

1 Verification and Validation Team Roles 5
2 RealmForge System Test Requirements Traceability Summary 33
3 Symbolic Constants Used in the Verification and Validation Plan 36

List of Figures

[Remove this section if it isn't needed —SS]

1 Symbols, Abbreviations, and Acronyms

symbol	description
T	Test

[symbols, abbreviations, or acronyms — you can simply reference the SRS [1] tables, if appropriate —SS]

[Remove this section if it isn't needed —SS]

This document outlines a plan for verification and validation (also denoted as VnV) of the RealmForge software. It includes a general summary of objectives and related documents, an overall plan discussing various verification and validation activities (concerning the SRS, design documents, the plan itself, and other artifacts), a set of system tests for both software functional and non-functional requirements, and finally a set of unit tests for software functional and non-functional requirements. The section outlining all unit tests (section 5) is currently incomplete (as it must be written taking the project's design document into account). The section will be updated following the completion of the project's detailed design document.

2 General Information

2.1 Summary

The software that will be tested is the RealmForge software, a game in the roguelite genre (features randomized runs with most progress resetting on defeat with some permanent upgrades) focused on expanding a central hub to aid in combat encounters. The system consists of many components including the Configuration Menu (allows tester to vary certain system variables to alter gameplay), Physical controls (handling input from physical devices like keyboards or game controllers), the Combat System (handling enemy encounters, attacks, and combat outcomes), a Central Hub (a hub to produce resources to fuel combat with temporary and permanent upgrades), a Map (allows user to travel to destinations and encounters) and a User Interface (menus and HUD elements). The following sections give a high-level overview of the rest of our VnV plan.

2.2 Objectives

The verification and validation plan objective is to build confidence in the functional sustainability of the software product. Systems tests are created to ensure the functional, nonfunctional and safety requirements are measured and quantified; this provides evidence that fit criteria are met and present in the account of the game. The system tests also reinforce the behavior of the game so that the requirements and other elements outlined in the SRS, Design Document and other future documents are tracked.

The VnV plan also makes sure that the game gets as full testing coverage as possible within the boundaries of the project; this provides a great steppingstone for CI implementation. Specific tools such as GDUnit4 and GDLint can implement the system tests (appropriate ones) to maintain coding standards, and requirement completion.

Other testings methods are also used to improve the quality of the software documentation and product. Code reviews, requirement reviews and document reviews are done by members within the team and peers from other associations. Usability testing will be done so that testing and feedback provided by the major stakeholder that are not in the capstone social group. Usability testing will help give direction on the requirements in order to create more or modify existing ones based on the results. All in all, this solidifies and strengthens the documentation and requirements which in turn improves the software product. Overall, the verification and validation plan contributes to the security in making sure the functional of the software is maintained and correctly resolved. The VnV plan gives a certain confidence level for quality and correctness of the final product.

2.3 Custom Documents

The Custom Documents the project will involve are a Feature Oriented Domain Analysis Report and a Usability Report.

A feature oriented domain analysis is an analysis showing the similarities and differences for a family of software products. It identifies common requirements and variabilities between members of a family of products, with variation parameters for each variability. Tables and/or diagrams of features can be used to display this.

A usability report is a systematic, rigorous and quantified report summarizing the results of usability testing plans. These results include metrics collected from game sessions, and user feedback about game design, issues/bugs, and user experience recommendations.

2.4 Relevant Documentation

- **Problem Statement and Goals** [2]: This document is relevant as it outlines the initial outline of RealmForge.
- **Development Plan** [3]: This document outlines the main goals, objectives, and stretch goals for the system. This informs all future development decisions, including this document.
- **System Requirements Specification (SRS)** [1]: This document outlines the functional and non-functional requirements for the system. The requirements listed inform the verification and validation efforts of this document.
- **Hazard Analysis** [4]: This document outlines safety and security requirements of the system, as well as listing the different potential failure modes of the system with causes and recommended action.
- **Feature Oriented Domain Analysis (FODA)** [**CITATION**]: This custom document outlines the variabilities within a software product line, which informs what common or unique aspects need to be verified in each member of the software family.
- **Usability Report** [**CITATION**]: This custom document outlines the results of usability testing. These results will be important when making any considerations about the system.
- **Module Guide (MG)** [5]: This document will be a reference for system components, informing important modules to test.
- **Module Interface Specification (MIS)** [6]: This document will help guide and define function, and therefore unit testing, in the system. This informs tests within the current document.

[Reference relevant documentation. This will definitely include your SRS and your other project documents (design documents, like MG, MIS, etc). You can include these even before they are written, since by the time the project is done, they will be written. You can create BibTeX entries for your documents and within those entries include a hyperlink to the documents. —SS]

Citation Example of SRS[1]

[Don't just list the other documents. You should explain why they are relevant and how they relate to your VnV efforts. —SS]

3 Plan

This section outlines the overall plans for the verification and validation of the system. This includes a list of team members and a description of their work on this document, verification of documents and the system, validation of the software, and the tools we will use for these tasks.

3.1 Verification and Validation Team

All members will contribute to and complete the verification and validation of RealmForge. Each team member has been assigned specific sections of the VnV Plan and is responsible for developing system tests related to their assigned areas, such as configuration and physical controls, combat, central hub functionality, overall gameplay and map navigation, and user interfaces.

Team members will also be assigned responsibility to review sections completed by other team members to ensure that tests align with the project requirements that have been outlined in the SRS documentation.

Alongside team members, the project supervisor, Dr. Welch, and teaching assistant, Rashid Bhuiyan, will also be involved throughout the project to provide continuous feedback on the overall project scope, deliverables, and the verification and validation process for RealmForge.

The following table outlines each member's primary responsibilities for the verification and validation process:

Team Member	Role
Andreas Sideris	SRS verification, design verification, and functional and nonfunctional user interface system tests.
Andy Huynh	VnV Plan verification, functional and nonfunctional system tests for configuration and physical controls, and the system test traceability table.
Owen Johnson	Implementation verification and functional and nonfunctional combat system tests.
Christopher Ludwig	Software validation and functional and nonfunctional central hub system tests.
Humna Saeed	VnV team documentation, automated testing and verification tools, and functional and nonfunctional system tests for overall gameplay and map functions.
Dr. Welch (Project Supervisor)	Provide feedback on the verification and validation process and review the overall feasibility of the project plan.
Rashid Bhuiyan (Teaching Assistant)	Provide feedback through regular TA meetings to discuss project deliverables, review documentation, and assess the overall completeness of the VnV Plan.

Table 1: Verification and Validation Team Roles

3.2 SRS Verification

The approaches listed below will be used to verify the software requirements specification for completeness, correctness, verifiability, avoidance of ambiguity, and traceability, among other relevant attributes. These methods are selected to provide confidence in the document, while also being feasible in the context of the project. Further verification activities would likely require a higher time investment for the RealmForge development team, which would be infeasible in conjunction with other project work, and other required courses.

Verification Approaches:

- **Course Peer Review and Update Tracking:** The SRS document was reviewed by another Capstone team, against the rubric and success criteria provided by the course. Suggestions and feedback provided by this peer review are tracked and maintained by GitHub issues, and suitable changes to address the feedback provided will be implemented before the document's Revision 1 completion. The team that conducted the peer review of the document may be contacted to further clarify their feedback, to ensure necessary changes are made.

- **Structured Team Requirements Review:** Prior to the completion of the SRS for the project's Revision 1, the RealmForge development team will conduct a structured review of the document, considering various qualities it must exhibit (completeness, correctness, etc. as listed above and in the course SRS checklist). This review will be a live team meeting, where team members will walk through each section of the document and verify its contents against the provided course SRS checklist, and the SRS document's rubric. This review will be conducted following all required document updates, such as those motivated by peer review feedback, TA feedback, course instructor feedback, or supervisor feedback. Any further required updates found during the review will be tracked and maintained in the team's GitHub repository and project (using issues).

- **Asynchronous Supervisor Review:** Since the first revision (Revision 0) of the SRS has been completed, our project supervisor Dr. Charles Welch will conduct an asynchronous review of the document, to provide feedback for the team to improve its contents. To respect Dr. Welch's time and ensure that critical feedback is produced, this review will focus on certain critical sections of the document, selected by the RealmForge development team. Dr. Welch will provide feedback in-person or via Microsoft Teams chats or video calls, depending on what he prefers.

3.3 Design Verification

The approaches listed below will be used to verify the project's software design document (once it is completed for each revision) with respect to course

standards, checklists, the document's implementation feasibility, and its effectiveness in providing a design that satisfies the SRS's requirements (among other relevant qualities). These methods are selected to provide confidence in the document, while also being feasible in the context of the project (as mentioned similarly for the verification of the SRS).

Verification Approaches:

- **Course Peer Review and Update Tracking:** The design document's initial revision (Revision -1) will be reviewed by another Capstone team, against the rubric and success criteria provided by the course. Suggestions and feedback provided by this peer review are tracked and maintained by GitHub issues, and suitable changes to address the feedback provided will be implemented before the completion of the document's next two revisions (0 and 1). The team that conducted the peer review of the document may be contacted to further clarify their feedback, to ensure necessary changes are made.
- **Structured Team Design Document Reviews:** Prior to the completion of the software design document for each of its revisions (-1, 0, and 1), the RealmForge development team will conduct a structured review of the document, considering various qualities it must exhibit (as discussed in course lectures, rubrics, and checklists focused on the design document). These reviews will involve live team meetings, where team members will walk through each section of the document and verify its contents against the provided course design document checklist, and the document's associated rubric. Prior to the completion of the document for its final two revisions (0 and 1), these reviews will be conducted following all required document updates, such as those motivated by peer review feedback, TA feedback, course instructor feedback, or supervisor feedback. Any further required updates found during each review will be tracked and maintained in the team's GitHub repository and project (using issues).

3.4 Verification and Validation Plan Verification

The verification and validation plan is an artifact in itself; it too must be verified.
Internal reviews for the verification plan are as follows:

- Peer reviews should be collected from classmates, another capstone group(s), and teaching assistants; these are formatted into GitHub issues. The suggestions and critiques should then be responded to from a project member with another GitHub issue that lays out the actions and changes required; these issues are labeled as "feedback response".
- Each member should create GitHub issues for their written sections. The issues should be updated and maintained to accurate feedback and response.
- Each team member will have an evenly divide number of sections and system tests corresponding to the document to look over such that the VnV plan will be completely reviewed. Each team member should note the section they are reviewing via the corresponding issue created.
- Mutation testing that introduces small defects to the requirements, test cases, and traceabilty links in order to determine if the VnV plan verfication can detect actual documentation faults

Below is a checklist to ensure the VnV plan is correct and complete

- ☐ Does the VnV plan verify all functional requirements
- ☐ Does the VnV plan verify all nonfunctional requirements
- ☐ Does the VnV plan verify all safety requirements
- ☐ Do all the outputs match the system tests desired output
- ☐ Does the system test cover all requirements listed in the SRS
- ☐ Do all system tests provide complete traceability
- ☐ Do all traceability links work
- ☐ Are all sections written with correct grammar
- ☐ Have all team members peer reviewed at least one of each of the other team member's work and the work is satisfactory
- ☐ Have all peer review (from other teams) issues been resolved

- ☐ Do all references, acronyms, and internal constants are properly detailed and linked
- ☐ Does the document cover all points provided in the respective checklists and rubric

3.5 Implementation Verification

The implementation verification will ensure that the implementation of RealmForge's codebase adheres to the SRS specifications set out. A combination of unit testing, static analysis, and regular code inspections.

Unit tests will be carried out using GDUnit4 [CITATION] as listed in this document. Passing these unit tests will verify if functionality has changed in any unintended way during refactoring a PR is allowed to be merged into their parent branch.

Static analysis will be carried out by GDLint [CITATION] throughout development to ensure a consistent code quality and style (as described in the Development Plan [CITATION]).

All code written will be reviewed by at least one other team member in a routine code inspection before merging. This will ensure not only that the code is of proper quality, but will ensure that the team is all on the same page and nothing takes anyone by surprise.

[You should at least point to the tests listed in this document and the unit testing plan. —SS]

3.6 Automated Testing and Verification Tools

RealmForge will implement the use of GitHub Actions to support continuous integration throughout development. Automated testing will be used to check changes in the project's code and help identify any major issues before changes are merged into the main branch.

If Godot is used for development, the team will consider implementing GdUnit4 for unit testing, along with GdUnit4 Coverage to assess how much of the code is covered by automated tests. This will allow for testing of individual scripts and game logic throughout the development process. The results can also

help identify areas that may require refactoring, fixes, or additional testing. Another GDScript tool the team is considering is GdLint, which can be used to maintain overall code quality and identify potential issues throughout development.

These testing and verification tools will allow for automated checks, unit testing, code coverage, and code quality checks. Code coverage results can be summarized in reports to help the team identify areas that require further testing. Automated testing may also be applied to pull requests before they are submitted for team review.

The testing and verification tools are subject to change as development progresses and further aspects of the game engine are explored. The team may also consider using additional external Godot modules and libraries as needed based on the project requirements.

3.7 Software Validation

The primary mechanism of validating RealmForge's software will be through extensive usability testing and playtesting, having users interact with the system, providing feedback according to our Usability Testing Plan found at **LINK THE PLAN**. Supplementary methods to usability testing and playtesting for software validation are:

- **Supervisor and Stakeholder Reviews:** Demonstrations of the project will be held throughout its lifecycle with course facilitators, key stakeholders (peers, potential users, etc.), and our project supervisor, Dr. Welch, such as during Proof of Concept and Rev0. We have also scheduled meetings to be held with Dr. Welch every other week to review anything in this project, including software validation.

- **Empirical Metric Validation:** Metrics (run duration, win/loss ratios, resource rates, etc.) will be collected with built-in tools will be collected during system use to create minimalistic quantitative date reports measuring engagement and other goals of the software. If tools allow, automated playthroughs of the game can be run to increase amount of empirical data.

- **Accessibility & Safety Validation:** Observational checks during playtesting alongside the execution of our UI system test suite will

validate startup epilepsy warnings (**SSR-006**), colourblind-accessible filters (**SSR-009**). Local save data recovery (**SSR-003**, **SSR-004**) will be validated through our functional testing.

4 System Tests

[There should be text between all headings, even if it is just a roadmap of the contents of the subsections. —SS]

4.1 Tests for Functional Requirements

[Subsets of the tests may be in related, so this section is divided into different areas. If there are no identifiable subsets for the tests, this level of document structure can be removed. —SS]

[Include a blurb here to explain why the subsections below cover the requirements. References to the SRS would be good here. —SS]

4.1.1 Configuration Tests for Functional Requirements

The area of testing below will focus on on the configurations aspect of RealmForge. The configurations are important because they control the overall dynamic experience of the game. Ensuring these tests work is a key component to allow users to experiment, test, and provide feed for usability.

1. **SFT-1.1** Test Player Health Configuration

   Control: Manual, Dynamic

   Initial State: The base increase for player health is +/- 0.

   Input: Change in the range control of the configuration health option.

   Output: The displayed health change is equal to the increment increase of the range control.

   Test Case Derivation: The expected output is justified based on the SFR-006 in the SRS document.

   How test will be performed:
   1. User Enter the configuration menu

2. Selects the player health option
3. Raise / Lower value via control dial
4. User begins game
5. User enters sees health on UI screen equal to health changed
6. Verify the displayed health is the same setting
7. User enters combat
8. User is attack by enemy
9. Verify the actual health pool is the amount displayed

2. **SFT-1.2** Test Resource Reward Management Configuration

   Control: Automatic

   Initial State: The base bonus amount of resources gathered is +/- 0.

   Input: Change in the current bonus resource is input value.

   Output: The reward health change is equal to the increment increase of the range control.

   Test Case Derivation: The expected output is justified based on the SFR-007 in the SRS document.

   How test will be performed:
   1.User Enter the configuration menu
   2. Selects the bonus resource option
   3. User enters combat
   4. User is successful in combat
   5. Verify the received award amount equal to Normal Amount and Bonus Amount

3. **SFT-1.3** Test Upgrade Cost Configuration

   Control: Manual, Dynamic

   Initial State: The base of all purchasable upgrades are set to their default value.

Input: Change the base upgrade cost from configuration before start of the game.

Output: The displayed and actual purchase cost is equivalent to the configured base value.

Test Case Derivation: The expected output is justified based on the requirement that the game shall have a configuration that changes the cost of each purchasable upgrade (SFR-008).

How test will be performed:
1. Users enters configuration menu
2. User selects upgrade cost option
3. User increases/decreases the cost with the provided control element
4. User begins a new game
5. User attempst to purchase an upgrade
6. Verify the displayed cost is equal to the configured value
7. Verfiy the amount of resources deducted equals the configured cost.

4. **SFT-1.4** Test Configuration Locked

Control: Manual, Dynamic

Initial State: The game has been configured with a game session active.

Input: Try to modify the configuration through various methods.

Output: No change in the configured values.

Test Case Derivation: The expected output is justified based on the requirement that the game shall prevent the user from changing the game's configurations until starting a new game (SFR-009).

How test will be performed:
1. User enters configuration menu
2. User modifies one or more configuration setting
3. User starts game
4. User begins gameplay and tries to navigate to configuration menu again (configuration display menu)
5. User attempts to change values again

6. Verify the configuration controls are removed or disabled.
7. Verify the original configuration values are unchanged.

5. **SFT-1.5** Test Configuration Confirmation Prompt

   Control: Manual, Dynamic

   Initial State: User has finished modifying all the desired configuration values.

   Input: The configuration setup is saved and displayable.

   Output: A confirmation prompt is shown after the user is leaving the menu and prepared to start the game.

   Test Case Derivation: The expected output is justified based on the requirement that the game shall give a prompt for confirmation once the user has finished setting up the game's configurations (SFR-010).

   How test will be performed:
   1. User enters configuration menu
   2. User modifies configuration settings
   3. User finishes the setup
   4. User leaves the configuration menu
   5. Verify the confirmation prompt appears
   6. Verify the contains the requirement UI elements
   7. User selects "Confirm" (or any option name to proceed)
   8. Verify the game has saved the configuration setup.

6. **SFT-1.6** Test Configuration For Controller device

   Control: Manual

   Initial State: Input device configuration is set to default keyboard.

   Input: Switch to controller and vice versa afterwards.

   Output: The game accepts inputs from only the controller after configuration setting and only keyboard and mouse after changing back.

Test Case Derivation: The expected output is justified based on the requirement that the game shall allow the user to configure the input device before starting a new game (SFR-003, SFR-004, SFR-005).

How test will be performed:
1. User enters configuration menu
2. User navigates to input device option
3. User selects "Controller" via the UI toggle
4. User exits configuration menu.
5. Verify that controller reponse works. Also verify keyboard response does not work.
6. Repeat steps 1-2. User selects "Keyboard & Mouse" via UI toggle
7. Verify that keyboard reponse works. Also verify controller response does not work.

4.1.2 Central Hub Tests for Functional Requirements

The system tests below are planned to verify the implemented system as a whole against functional requirements related to the Central Hub component of the system.

1. **SFT-2.1** Permanent Upgrades to Central Hub

   Control: Manual

   Initial State: The current game state is focused on the Central Hub with a permanent upgrade available to be obtained.

   Input: The user unlocks the permanent upgrade, and the current run is then ended.

   Output: A permanent upgrade to the Central Hub is now associated with the corresponding player profile which unlocked it, and is always available to that profile in future runs.

   Test Case Derivation: This system test verifies that upgrades which are meant to be permanent are maintained between runs and are not unintentionally removed from the player profile.

   How test will be performed: The system tester will launch the RealmForge software, and begin a run or resume an existing one. Once

they are in a run, they will trigger a permanent upgrade unlock and then end the run either by intentionally exiting or dying. If the upgrade is supposed to take effect immediately, the system tester shall continue their run to see it's effect and then end the run. They will then start a new run and see if the previously unlocked upgrade is still unlocked and working as intended.

2. **SFT-2.2** Resource Generation from Central Hub

   Control: Manual

   Initial State: The RealmForge software has been launched on the user's device and initialized into a game state where they are starting a combat encounter.

   Input: The user selects to enters the combat encounter and performs any action that requires resources.

   Output: The system initializes resource count variables to the starting amount corresponding with Central Hub progress, updating it in accordance to the resource generation rate of the Central Hub and maintaining an accurate count as user actions expend resources simultaneously. This resource count should be constantly displayed to the user.

   Test Case Derivation: This test verifies an essential mechanic of the core gameplay loop of RealmForge, ensuring that resource generation from the Central Hub works as intended, giving the user access to the correct number of resources in accordance to upgrades to the Central Hub.

   How test will be performed: The system tester will launch the RealmForge software, and begin a run or resume an existing one. Once they are in the run, they will note down the current state of the Central Hub and calculate the correct amount of starting resources and the rate of generation of each resource type. Then the system tester shall enter a combat encounter, verifying the starting resource count against the calculated number as well as recording the rate at which they gain resources precisely to verify against the other calculated numbers.

3. **SFT-2.3** Temporary Upgrades to Central Hub

   Control: Manual

   Initial State: The RealmForge software has been launched on the user's device and the current game state is initialized to focus on the Central Hub with a temporary upgrade available to be obtained.

   Input: The user obtains the temporary upgrade and continues playing the game until their run ends and then begins a new run.

   Output: The Central Hub is upgraded for the rest of the run, altering how it functions according to the upgrade selected, and then is reinitialized with none of the temporary upgrades when a new run is started after the initial run has ended.

   Test Case Derivation: This system test verifies that temporary upgrades do affect how the Central Hub functions as well as not being initially present in the Central Hub when a new run is started.

   How test will be performed: The system tester will launch the RealmForge software, and begin a run or resume an existing one. Once they are in a run, they will trigger a temporary upgrade unlock and then continue their run, tracking that the upgrade to the Central Hub has altered it in exactly the way intended for the rest of the run. Then the system tester will end the run either by intentionally exiting or dying. They will then start a new run and see if the previously unlocked upgrade is no longer unlocked initially.

4. **SFT-2.4** Selecting Upgrades for the Central Hub

   Control: Manual

   Initial State: The RealmForge software has been launched on the user's device and the system is initialized to a game state where the user should be able to upgrade the Central Hub.

   Input: No input is necessary once at the initial state

   Output: The system shall always display atleast one upgrade to the user and all upgrades shall contained detailed descriptions for exactly what they do and the cost to obtain them.

Test Case Derivation: This system test verifies the user will never get stuck with no options for upgrades to select and all upgrades always contain the necessary details for the user to make informed decisions and leading to a better gameplay experience.

How test will be performed: The system tester shall play through many runs and track whether the intended output for displaying upgrade options is displayed as well as recording what they think each upgrade does according to the description and if they ever thought the cost of an upgrade was different from what it was.

5. **SFT-2.5** Rejecting Upgrade Unlock Due to Insufficient Funds

   Control: Manual

   Initial State: The RealmForge software has been launched on the user's device and the system is initialized to a game state where the user should be able to upgrade the Central Hub.

   Input: The user attempts to unlock an upgrade in which they do not meet the requirements for unlocking.

   Output: The system rejects the unlock attempt and notifies the player they do not meet the requirements for unlocking the upgrade

   Test Case Derivation: The system test ensures that the upgrade unlock mechanic is working as intended and does not allow users to unlock all upgrades for free, eliminating all need for thought and strategy for upgrade selection.

   How test will be performed: The system tester will launch the RealmForge software, and begin a run or resume an existing one. Once they are in the run, they will play until reaching a game state where they can unlock upgrades for the Central Hub and one of the available upgrades is one in which they do not meet the requirements for. They will then attempt to unlock that upgrade and note down how the system responds.

6. **SFT-2.6** Verifying Game State Transition to the Central Hub

   Control: Manual

Initial State: The RealmForge software has been launched on the user's device and the current game state has the option to switch to the game state focused on the Central Hub.

Input: The user triggers the game state transition to focus on the Central Hub.

Output: The system transitions from one game state to the next, verifying a game state change did occur.

Test Case Derivation: The system test verifies that game state transitions are working in our system and bugs will not lock users out of progressing forward.

How test will be performed: The system tester will launch the RealmForge software, and begin a run or resume an existing one. Once they are in a run, they will play the game normally but they should keep close track of game information. When a game state transition should occur, the tester will verify the state change did happen and that no data was corrupted during the transition.

4.1.3 Area of Combat Tests for Functional Requirements

The system tests listed below are planned to verify the implemented system as a whole against functional requirements related to the area of combat component of the system. These requirements can be found in the SRS [1], under section S.2's functional requirements for the 'Combat System'.

1. **SFT-3.1** System Initialization

   Control: Manual

   Initial State: The RealmForge software has been launched on a compatible device, and initialized to a game state where the player is selecting a combat encounter.

   Input: A valid input is pressed using a valid input device (keyboard and mouse, or controller) in order to begin an encounter.

   Output: The system transitions its internal state to a combat encounter, and displays the encounter to the player.

Test Case Derivation: This system test verifies that the combat system can successfully load and display the relevant information within a combat encounter. This functionality is required for the system to allow proper visual feedback within a combat encounter.

How test will be performed: The system tester will launch the RealmForge software, and configure it to enter a game state where a combat encounter is selected from the valid locations on the map. Visual inspection takes place to ensure the combat encounter is properly initialized (player/enemy sprites and health displayed properly), and the displayed values will be checked against the proper values associated with the player/enemies.

2. **SFT-3.2** System Feedback

Control: Manual

Initial State: The RealmForge software has been launched on a compatible device, and initialized to a game state where the player is in a combat encounter and and action is about to take place.

Input: A valid player action is selected using a valid input device (keyboard and mouse, or controller), or an enemy proceeds with an action.

Output: The system displays a visual that signifies an action has taken place, and transitions its internal state depending on the result of the action taken. All relevant measured statistics are updated accordingly.

Test Case Derivation: This system test verifies that the combat system can successfully update the display of the relevant information within a combat encounter and update this information as necessary. This functionality is required for the system to allow change to the proper visual feedback within of a combat encounter, allowing an encounter to visually progress.

How test will be performed: The system tester will launch the RealmForge software, and configure it to enter a game state where either the player can select an action or an enemy is about to do an action. The tester along with the test suite will then verify that the

system successfully transitions the information and corresponding visuals following an action. The test will be repeated as discussed in SNFT-3.2 to ensure variability.

3. **SFT-3.3** Display Combat Outcome

   Control: Manual

   Initial State: The RealmForge software has been launched on a compatible device, and initialized to a game state where the player's current combat encounter is over.

   Input: After a player/enemy action is input, the system detects either the player's or enemies' health decreasing to or below 0, corresponding to either a win or a loss (the case where both player and enemy health are depleted counts as a loss).

   Output: The system transitions its internal state from a combat state to a results state. If the player won, the display transitions to a results screen and presents the player rewards, ensuring they are properly awarded. If the player lost, the display transitions to a game session restart screen.

   Test Case Derivation: This system test verifies that the combat system can successfully detect win/lose conditions within a combat encounter and transition game state/display as necessary (results screen and rewards, or game session restart screen). This functionality is required to ensure that the player actually keeps their encounter rewards on victory. This functionality is also required for the system to allow transition out of a combat encounter, preventing the player from being softlocked [GLOSSARY CITATION NEEDED] and requiring the system to restart.

   How test will be performed: The system tester will launch the RealmForge software, and configure it to enter a game state where the player has just won a combat encounter. The tester will then verify that the system successfully transitions the display output to the results screen following the end of a combat encounter. The test will be repeated for a combat loss.

4.1.4 User Interface Tests for Functional Requirements

The system tests listed below are planned to verify the implemented system as a whole against functional requirements related to its user interface component. These requirements can be found in the SRS [1], listed in section S.2's functional requirements for the 'User Interface System', as well as in section S.2's Safety and Security Requirements. Traceability relationships between all system tests and covered requirements are provided in the traceability table in section 4.3 of this document.

1. **SFT-4.1** Main Menu Interface

   Control: Manual

   Initial State: The RealmForge software has been launched to its initial menu on a compatible device.

   Input: Each available main menu option is selected ('start game', 'settings', and 'exit') via an input device (keyboard and mouse, or a controller). Once a main menu option has been selected, an input is also provided to return to the main menu from these options' screens (game start menu, configurations menu, etc.).

   Output: The system transitions its internal state and displayed visuals to the selected menu after each selection input. For example, when 'settings' is selected, the display transitions to the configurations menu, and when an input is received to return to the main menu, the system transitions back to the main menu. This is also performed for other options available from the main menu.

   Test Case Derivation: This system test verifies at a basic level that the system's main user interface (the main menu) can successfully receive user inputs and transition between states as necessary. This functionality is required for the system and its interface to receive other inputs and provide the necessary outputs during gameplay and other uses of the software.

   How test will be performed: The system tester will launch the RealmForge software, and verify that the system successfully transitions the display output between each menu provided by the options

available from the main menu, and back to the main menu when prompted. This will be verified for each major type of input device (keyboard and mouse, and controller).

2. **SFT-4.2** Game Session Restart Interface

   Control: Manual

   Initial State: The RealmForge software has been launched on a compatible device, and initialized to a game state where the player's current game run has ended (a run is an attempt at completing the game, which can end following the player successfully completing the game or failing during an attempt).

   Input: The various options available to the user following the end of a game run are selected (including restarting the run in the same saved game session / save file, or starting a new save file) via an input device (keyboard and mouse, or a controller).

   Output: When the option to restart the game run in the same save file is selected, the system transitions its internal state and display output to the initial game scene displayed during a game run, with the user's previous save data included where appropriate. Similarly, when the option to start a new game run in a new save file is selected, the system transitions to the interface used to create a new save file.

   Test Case Derivation: This system test verifies that the interface provided by the system following the end of a game run can successfully receive user inputs and transition between its next possible states as necessary (restarting in the same save file, or creating a new save file). This functionality is required for the system to allow users to continuously attempt runs of the game (which is a common gameplay loop for video games in the roguelite genre, as RealmForge is), or create a new save file when needed.

   How test will be performed: The system tester will launch the RealmForge software, and configure it to enter a game state following the end of a player's game run. The tester will then verify that the system successfully transitions the display output between each interface

provided by the options available following the end of a run (restarting in the same save file, and creating a new save file). This will be verified for each major type of input device (keyboard and mouse, and controller).

3. **SFT-4.3** Display of Flashing Lights Warning

   Control: Manual

   Initial State: The RealmForge software has been installed and deployed/compiled on a compatible device.

   Input: The RealmForge software is launched on a compatible device.

   Output: Following launch, the system displays a warning to the user that gameplay may result in the display of various images and colours that can cause seizures for individuals with photosensitive epilepsy (despite meeting standard guidelines, as outlined in SNFT-5.3). The warning clearly states that users with photosensitive epilepsy or a family history of the condition should consult their physician before playing the game.

   Test Case Derivation: This system test verifies that the system successfully displays a required warning for users with photosensitive epilepsy (or those with a family history of the condition), to ensure that these users do not continue gameplay without awareness of how it could affect their condition. While RealmForge is required to meet certain accessibility standards to ensure it does not display any flashing images that could harm these individuals (as outlined in SNFT-5.3), it is important to recommend that they consult their physician first before playing the game in the interest of caution.

   How test will be performed: The system tester will launch the RealmForge software, and verify that the correct flashing images/lights warning is displayed by the system.

4.2 Tests for Nonfunctional Requirements

4.2.1 Configuration Tests for Nonfunctional Requirements

4.2.2 Area of Testing1

Title for Test

1. test-id1

   Type: Functional, Dynamic, Manual, Static etc.

   Initial State:

   Input/Condition:

   Output/Result:

   How test will be performed:

2. test-id2

   Type: Functional, Dynamic, Manual, Static etc.

   Initial State:

   Input:

   Output:

   How test will be performed:

4.2.3 Area of Combat Tests for Non-Functional Requirements

The system tests listed below are planned to verify the implemented system as a whole against non-functional requirements related to the area of combat component of the system. These requirements can be found in the SRS [1], under section S.2's Non-Functional Requirements and Safety and Security Requirments.

1. **SNFT-3.1** Action Selection Feedback

   Type: Dynamic, Manual

Initial State: The RealmForge software has been launched on a compatible device, and initialized to a game state where the player is in a combat encounter. In this state the player can select an action.

Input/Condition: The system proceeds to a state where the player can select an action through inputs made using a valid input device (keyboard and mouse, or controller) This can be on first entrance to a combat encounter, or action returning to the player after enemy action(s).

Output/Result: The system displays the various action options the player has access to (with proper descriptions), and properly signifies to the player which actions are or are not valid, giving the player proper feedback. The visuals for valid/not valid actions are distinct from each other (following required accessibility guidelines).

How test will be performed: The system tester will launch the RealmForge software, and initialize its state to a combat encounter. The tester will visually inspect the combat UI to evaluate visual indication of action validity and proper description of actions, and then will select the action using a valid input device as a player would, verifying the validity of the actions (visual indication should match what the system allows). This is repeated as necessary for the various player actions present in the UI.

2. **SNFT-3.2** Combat Action Feedback

   Type: Dynamic, Manual

   Initial State: The RealmForge software has been launched on a compatible device, and initialized to a game state where the player is in a combat encounter. In this state the player can select an action (either offensive or defensive), or is awaiting an enemy action.

   Input/Condition: A combat action (attack, heal, etc.) is selected using inputs to the software using a valid input device (keyboard and mouse, or controller), or the system is transitioned to a state where enemy action is awaited.

   Output/Result: The system completes the player/enemy action if valid, displays the proper visual for the action taken, and updates any

visual indicators or stat changes (e.g. health). These action visuals are distinct from each other (following required accessibility guidelines).

How test will be performed: The system tester will launch the RealmForge software, and initialize its state to a combat encounter. The tester will then select an action from the combat UI using a valid input device as a player would. They will verify that for each action selected, the correct distinct visual corresponding to the action is displayed, and that any visual indicators of system changes are displaying the proper updated information. This is repeated as necessary for the various player/enemy actions present, both offensive and defensive.

3. **SNFT-3.3** Enemy Encounter Variation

   Type: Dynamic, Manual

   Initial State: The RealmForge software has been launched on a compatible device, and initialized to a game state where the player is in a combat encounter.

   Input/Condition: A combat action that affects an enemy is selected using a valid input device (keyboard and mouse, or controller).

   Output/Result: The system completes the player action if valid, displays the proper visual for the action taken, and updates any visual indicators or stat changes (e.g. health). These action visuals are distinct from each other (following required accessibility guidelines).

   How test will be performed: The system tester will launch the RealmForge software, and initialize its state to a combat encounter. The tester will visually inspect the enemy sprites and visible stats to evaluate visual indication of enemy variability. They will then select an action using a valid input device as a player would, and evaluate if enemies have varying statistics that are hinted at through their visuals (e.g. the one with a shield has higher defense and loses less health from an attack). This is repeated as necessary for the relevant enemies present in the encounter, and repeated to find at least MINIMUM_VARIABLE_ENCOUNTERS encounters with varying aspects to prove enemy variability.

4.2.4 Central Hub Tests for Nonfunctional Requirements

The system tests below are planned to verify the implemented system as a whole against nonfunctional requirements related to the Central Hub component of the system.

1. **SNFT-4.1** Sufficient Variety to Upgrades in Central Hub

   Type: Manual

   Initial State: The RealmForge software has been launched on the user's device and they have started a new run or resumed an existing one.

   Input/Condition: The user plays through multiple runs of the game, unlocking temporary and permanent upgrades from the Central Hub as they see fit.

   Output/Result: The user will have their Central Hub upgraded and will report their opinion on the variety of upgrade options given to them.

   How test will be performed: The system tester shall play through many runs of the game, noting down the number of upgrade options available to them whenever they are give the option to unlock upgrades. They should also be recording how different and impactful each upgrade seems to help measure variety between upgrades.

2. **SNFT-4.2** Available and Unavailable Upgrade Test for Central Hub

   Type: Manual

   Initial State: The RealmForge software has been launched on the user's device and the system is initialized to a game state where the user should be able to upgrade the Central Hub.

   Input/Condition: No input is required for this test

   Output/Result: The user will report if they were ever confused whether an upgrade as available for upgrade or not.

How test will be performed: The system tester shall launch the RealmForge software and begin a new one or resume an existing one. They will play through these runs and every time an upgrade is available, they should confirm if their first thought whether an upgrade was available or not, recording every instance in which they were wrong.

4.2.5 User Interface Tests for Non-Functional Requirements

The system tests listed below are planned to verify the implemented system as a whole against nonfunctional requirements related to its user interface component. These requirements can be found in the SRS [1], listed in section S.2's nonfunctional requirements (they are spread across multiple categories), as well as in section S.2's Safety and Security Requirements (some safety requirements presented in the document relate to nonfunctional qualities of the system). Tracebility relationships between all system tests and covered requirements are provided in the traceability table in section 4.3 of this document.

1. **SNFT-5.1** Game World Location Appearance Variety

   Type: Dynamic, Manual

   Initial State: The RealmForge software has been launched on a compatible device, and initialized to a game state where the overall game world is displayed during a gameplay session.

   Input/Condition: The game world is traversed using inputs to the software as a player would during gameplay (assuming the software is initialized to a state where the player is permitted to move to each world location), using a valid input device (keyboard and mouse, or a controller).

   Output/Result: The total game world displays at least DISTINCT_LOCATION_ICONS unique icons/symbols for the locations players can traverse to (these can be repeated across the game world as needed). A used symbol is distinct from others if it is visibly differentiated by colour (following required accessibility guidelines), or the images displayed (a mountain vs. a forest, for example).

How test will be performed: The system tester will launch the RealmForge software, and initialize its state to the overall game world (where each game world location is valid for travel). The tester will then traverse the game world using a valid input device as a player would, and verify that at least DISTINCT_LOCATION_ICONS distinct location symbols are displayed in the total game world, following the definition of distinct described above.

2. **SNFT-5.2** Game State Display Transitions

   Type: Dynamic, Manual

   Initial State: The RealmForge software has been launched on a compatible device, in its development environment.

   Input/Condition: The game is initialized to each generalized state it can enter (main menu, configuration menu, save data management menu, game world, and combat encounter) and each potential scene/menu transition for each state is executed via a valid input device (entering the configuration menu from the main menu, for example).

   Output/Result: The system completes and displays each prompted game scene transition in under TRANSITION_TIME seconds. During each transition, the system fades the display output to black (or another suitable intermediate screen), to communicate that the transition is occurring.

   How test will be performed: The system tester will launch the RealmForge software, and initialize its state to each generalized state it can enter (as discussed above). The tester will then prompt the system to execute each possible scene/menu transition from each state, and verify that each transition elapses at most TRANSITION_TIME seconds. This includes one transition for each time the game can transition to a new overall state (i.e. from the main menu to the save data management menu); granular transitions are not tested (such as the system transitioning from one state of an enemy encounter to another).

   The tester will also verify that the system displays a valid transition screen, as described above.

Since this test involves verifying qualities of many different transitions the system can execute, it is suitable for each transition to be verified a minimum of TRANSITION_TESTS times. This ensures that the test is feasible in the context of the project.

3. **SNFT-5.3** Review of Flashing Lights in Game Content

   Type: Static, Manual

   How test will be performed: The RealmForge software's source code will be reviewed by the RealmForge developers to ensure that any graphical displays or changes follow the WCAG 2.2 accessibility guidelines' guideline 2.3.1 [7]. This guideline mandates that presented content does not display any flashing images more than three times in any one second period, to reduce any risks to individuals with photo-sensitive epilepsy.

   Each module of the RealmForge software's source code will be verified against this guideline, during regular reviews held by the RealmForge development team following the completion of a new section of source code. Sections of source code that display graphical outputs to the user in a timely fashion will be specifically reviewed, to ensure that they do not display any excessively bright images that change more than three times in one second.

4. **SNFT-5.4** Review of Distinguishable Colours in Game Content

   Type: Static, Manual

   How test will be performed: The RealmForge software's source code will be reviewed by the RealmForge developers to ensure that any graphical indicators or interface elements/buttons used in the software follow the WCAG 2.2 accessibility guidelines' guideline 1.4.1 [7]. This guideline mandates that "color is not used as the only visual means of conveying information, indicating an action, prompting a response, or distinguishing a visual element" [7].

   Each module of the RealmForge software's source code will be verified against this guideline, during regular reviews held by the RealmForge development team following the completion of a new section of source

code. Sections of source code that display graphical outputs to the user that must be differentiated from other graphics (such as interface elements, system messages, etc.) will be specifically reviewed, to ensure that they provide a means of distinguishing these elements from others without relying on using colour.

4.2.6 Software Usability Tests

The usability tests listed below are planned to assist other primary tests in verifying the implemented system as a whole against requirements related to the user experience, as well as validate the RealmForge software with respect to the needs of its users. The tests' related requirements can be found in the SRS [1] in Section S.2. Tracebility relationships between all system tests and covered requirements are provided in the traceability table in section 4.3 of this document.

1. **SNFT-6.1** Primary System Playtests

   Type: Dynamic, Manual

   Initial State: A number of playtesters (game usability test participants) greater than or equal to MIN_TESTERS owning compatible devices have been contacted and selected to participate in testing. The RealmForge software has been installed and compiled on each participants' device.

   Input/Condition: Each participant will launch the RealmForge software on their device, and play through at least two game runs (an attempt at completing the game, from the beginning to completion/success or defeat, after which the user can start a new run) for each potential feature configuration provided by the game.

   Output/Result: Each participant will complete a survey (with questions available in 6.2 Usability Survey Questions). This survey will collect data from participants related to their experience when playing the RealmForge game, when configured with specific gameplay feature variations. This data is the primary output of the system playtests, and will be used to inform the development of the software. Participant experience data may be summarized using various charts and plots to better inform development.

How test will be performed: Each participant will play through at least two game runs for each of the following gameplay feature variation points provided by the software:

- Player Character Starting Health
- Rewards Gained Following an Enemy Encounter
- Central Hub Upgrade Costs
- Enemy Encounter Combat Pace

For each variation point, each participant will play through one game run with the configured feature set to its default state, and at least one game run with the configured feature set to an alternative state. This will allow participants to understand how their experience is positively or negatively impacted (in their own opinion) by the variation of each feature configuration, and find which configurations improve their experience overall (which they will then report via the survey). Participants will also be able to report other details related to their experiences playing the game, such as errors they detected, or suggestions they may have to improve gameplay.

4.3 Traceability Between Test Cases and Requirements

Table 2: RealmForge System Test Requirements Traceability Summary

System Test ID	Requirements Coverage
SFT-1.1	SFR-006
SFT-1.2	SFR-007
SFT-1.3	SFR-008
SFT-1.4	SFR-009
SFT-1.5	SFR-010
SFT-1.6	SFR-003, SFR-004, SFR-005
SFT-3.1	SFR-011, SFR-012, SFR-017
SFT-3.2	SFR-014, SFR-015, SFR-016

Requirement ID	Requirements Coverage
SFT-3.3	SFR-013, SSR-007, SSR-012
SNFT-3.1	SNFR-008, SSR-008
SNFT-3.2	SNFR-002, SNFR-005
SNFT-3.3	SFR-017, SNFR-002, SNFR-005
SFT-4.1	SFR-018
SFT-4.2	SFR-019
SFT-4.3	SSR-006
SNFT-5.1	SNFR-004
SNFT-5.2	SNFR-009
SNFT-5.3	SSR-006
SNFT-5.4	SSR-009
SNFT-6.1	SFR-006, SFR-007, SFR-008, SNFR-003
SFT-2.1	SFR-025
SFT-2.2	SFR-026
SFT-2.3	SFR-026
SFT-2.4	SFR-027, SFR-028, SFR-029
SFT-2.6	SSR-012
SNFT-4.2	SSR-008
SFT-2.5	SFR-030
SNFT-4.1	SNFR-001

5 Unit Test Description

TBD

References

[1] Aeolus Software (https://github.com/Aeolus-Software), "Realmforge system requirements specification," https://github.com/Aeolus-Software/RealmForge/blob/main/pdfs/SRS/SRS.pdf, Aeolus Software, 2026.

[2] ——, "Realmforge problem statement and goals," https://github.com/Aeolus-Software/RealmForge/blob/main/pdfs/ProblemStatementAndGoals/ProblemStatement.pdf, Aeolus Software, 2026.

[3] ——, "Realmforge development plan," https://github.com/Aeolus-Software/RealmForge/blob/main/pdfs/DevelopmentPlan/DevelopmentPlan.pdf, Aeolus Software, 2026.

[4] ——, "Realmforge hazard analysis," https://github.com/Aeolus-Software/RealmForge/blob/main/pdfs/HazardAnalysis/HazardAnalysis.pdf, Aeolus Software, 2026.

[5] ——, "Realmforge module guide," https://github.com/Aeolus-Software/RealmForge/blob/main/pdfs/Design/SoftArchitecture/MG.pdf, Aeolus Software, 2026.

[6] ——, "Realmforge module interface specification," https://github.com/Aeolus-Software/RealmForge/blob/main/pdfs/Design/SoftDetailedDes/MIS.pdf, Aeolus Software, 2026.

[7] World Wide Web Consortium (W3C), "How to Meet WCAG (Quick Reference): A Customizable Quick Reference to Web Content Accessibility Guidelines (WCAG) 2 Requirements," *W3C*, https://www.w3.org/WAI/WCAG22/quickref/, (accessed Nov. 1, 2026).

6 Appendix

This is where you can place additional information.

6.1 Symbolic Parameters

The following symbolic constants are used throughout this document:

Table 3: Symbolic Constants Used in the Verification and Validation Plan

Constant	**Value**	**Unit**
DISTINCT_LOCATION_ICONS	5	Unitless/Quantity
TRANSITION_TIME	4	Seconds
TRANSITION_TESTS	3	Unitless/Quantity
MIN_TESTERS	5	Unitless/Quantity

6.2 Usability Survey Questions

The following questions will be posed to participants of the RealmForge software's primary system playtests, as outlined in SNFT-6.1:

Primary System Playtest Survey:

1. During your gameplay session, which configurations of the player character's starting health were tested?

2. How was your experience impacted by varying this feature's parameters? Did increasing the player character's initial health cause the game to be more engaging and rewarding to play, or the opposite? How did reducing the player character's initial health impact your experience?

3. Which configuration of this feature that you tested resulted in the most engaging experience, in your opinion?

4. During your gameplay session, which configurations of the rewards gained following an enemy encounter were tested?

5. How was your experience impacted by varying this feature's parameters? Did increasing the rewards granted to you following a successful enemy encounter cause the game to be more engaging and rewarding to play, or the opposite? How did reducing the rewards granted impact your experience?

6. Which configuration of this feature that you tested resulted in the most engaging experience, in your opinion?

7. During your gameplay session, which configurations of the central hub's upgrade costs were tested?

8. How was your experience impacted by varying this feature's parameters? Did increasing the costs of upgrades available in the central hub cause the game to be more engaging and rewarding to play, or the opposite? How did reducing the costs impact your experience?

9. Which configuration of this feature that you tested resulted in the most engaging experience, in your opinion?

10. During your gameplay session, which configurations of the enemy encounter combat pace were tested?

11. How was your experience impacted by varying this feature's parameters? Did increasing the speed of enemy encounters cause the game to be more engaging and rewarding to play, or the opposite? How did reducing the speed impact your experience?

12. Which configuration of this feature that you tested resulted in the most engaging experience, in your opinion?

13. Did you encounter any errors, unexpected events, or unexpected displayed visuals during your game session? If so:

    (a) What occurred?

    (b) Which part of the game were you playing at the time?

    (c) What inputs did you provide?

14. Did any features of the game negatively impact your experience, or were any features frustrating (for example, if a certain menu was difficult to use)? If so, which features, and why?

15. Do you have any suggestions for new features that would improve your experience when playing the game, or modifications to existing features that would improve your experience?

7 Appendix — Generative AI Use Disclosure

7.1 AI Tools Used

- Anthropic Claude Sonnet 5.5
- Google Search AI Overview (Google Gemini)

7.2 How and Where Used

AI tools were used for assistance with LaTeX syntax and commands, during the development of sections SRS Verification and Design Verification, and when developing LaTeX formatting for the document as a whole. Specifically, AI tools were used for assistance in creating the LaTeX custom system test format macro used throughout the document, as well as the symbolic parameter table used in section 6.1. In these sections, AI tools were not used to generate any content text (LaTeX formatting code was generated), or wholly generate any of the ideas or conclusions presented.

7.3 Verification of AI-Generated Content

Any LaTeX syntax or code provided by AI tools was verified by testing compilation in LaTeX documents, and ensuring the resulting output was correctly formatted and matched the needs of the document. No ideas or text (outside of LaTeX formatting code) were newly generated by AI tools and presented in the document.

8 Appendix — Reflection

Reflection Questions Answered:

1. What went well while writing this deliverable?

2. What pain points did you experience during this deliverable, and how did you resolve them?

3. What knowledge and skills will the team collectively need to acquire to successfully complete the verification and validation of your project? Examples of possible knowledge and skills include dynamic testing knowledge, static testing knowledge, specific tool usage, Valgrind etc. You should look to identify at least one item for each team member.

4. For each of the knowledge areas and skills identified in the previous question, what are at least two approaches to acquiring the knowledge or mastering the skill? Of the identified approaches, which will each team member pursue, and why did they make this choice?

Andreas:

1. When writing this deliverable, our team worked well to divide the work required evenly, and ensure that all team members contributed to each important section of the document. This helped us to ensure that critical sections were developed by taking all team members' ideas and contributions into account. Specifically, instead of having one team member focus on writing a large swath of system tests, and making other members focus on most of the general plan sections of the document (SRS verification, etc.), all users contributed to the system tests, ensuring that our ideas and perspectives were combined to cover the system's requirements as much as possible.

2. One pain point I experienced during the deliverable was attempting to find a balance between feasibility and coverage when writing system tests. When creating a plan to test the system as a whole, our goal was to cover all requirements mandated in the SRS, and extensively test the system so that it is verified and validated as a professional engineering product. However, since the project's development must fit in an eight month scope (of which there are approximately five to six

months remaining), with verification activities likely being allotted a fraction of that time (to fit with other development activities), the plan for testing must be scoped to be feasible in the project's context. This balance was found by ensuring that the system tests I wrote minimally cover the requirements provided in the SRS, and any other system qualities that must be tested, but do not necessarily have to extend into a large, complex set of tests across a large sample size. For example, while primary playtests (usability tests) for the software are planned, the number of participants in these tests and the amount of testing they must participate in is given a conservative bound, to ensure feasibility. However, if extra time and resources are available in the project to complete more testing, the scope of the tests planned can be increased to ensure better coverage.

3. One skill I will personally need to acquire is a proficiency with unit testing Godot game engine code. Since Godot has been selected as the game engine that will be used for the development of the project's software, its capabilities for unit testing its GDScript language must be used to implement automated unit testing. Additionally, I will need to acquire the skill of executing automated unit tests of GDScript code with GitHub Actions or CI/CD pipelines, since it is required for the project.

4. For both skills above, the knowledge required to improve my proficiency could be found from video tutorials (on YouTube, for example), official Godot tutorials and documentation, or from being taught by other team members or peers with experience with Godot. In my case, I will primarily use the official Godot tutorials and documentation to learn these skills, as official sources tend to have the most accurate information for the exact version of the game engine we will use in the project. Additionally, not relying on others for instruction related to the skills (while help can still be acquired from team members or peers occasionally) can help me to efficiently learn the skills required asynchronously.

Humna:

1.

2.

3.

4.

Christopher:

1. Team communication went well during this deliverable, having meetings to be able to discuss what needs to be done for remaining sections and reviewing each other's work. Our virtual communication channels have also been very helpful for allowing us to stay in touch, having any issues receive quick responses from other team members.

2. A specific pain point during this deliverable was how time-constrained we felt while working on it. With only two weeks since the last deliverable in which the first week were filled with midterms for many of the team members and other large projects due at the same time, it was very busy and hard to manage the schedule. To resolve this, we committed to starting work as soon as there was a break from midterms and create a schedule for team members to complete sections by certain dates.

3. A skill necessary for the whole team would be understanding game architecture and common game coding styles. This will be important for the team so we all comply to common standards for game development and it will be significantly easier for team members to collaborate on closely related components. It will also be easier to revise each other's code and design the overall architecture to be efficient and easily understandable.

4. Two different methods for team members to acquire the knowledge/skills for understanding game architecture and common game coding styles would be to do research online from verifiable and trustworthy sources or to review these topics as a team with more experienced team members sharing their experiences and advice. Doing research is a good method to see what reputable sources and experienced experts have to say on the matter while team review sessions are good as we will all be learning the same standard to easily collaborate with each other in the future. I will use a mix of both, primarily doing research online as it can be hard to synchronize schedules to learn hard concepts together and there is lots of valuable information online.

Owen:

1.

2.

3.

4.

Andy:

1.

2.

3.

4.